

ASDIA: A stress-aware navigation application for individuals with Autism Spectrum Disorder

Adrien Cordero
student at INSA Toulouse
cordero@insa-toulouse.fr

Anthony Ferrere Hoareau
student at INSA Toulouse
ferrere-hoar@insa-toulouse.fr

Guilhem Callis
student at INSA Toulouse
callis@insa-toulouse.fr

Iness Nouiga
student at INSA Toulouse
nouiga@insa-toulouse.fr

Nabil Benkerdagh
student at INSA Toulouse
benkerdagh@insa-toulouse.fr

Notahiana Erwan Ramangason
student at INSA Toulouse
ramangason@insa-toulouse.fr

Thibault Eynard-Suarez
student at INSA Toulouse
eynard-suare@insa-toulouse.fr

Abstract—While navigation applications have become essential in most people’s daily life, none truly take into account the diverse and specific requirements of individuals with Autism Spectrum Disorder (ASD). Considering the multiple stress factors associated with navigation that strongly affect individuals with ASD, we propose ASDIA (Autism Spectrum Disorder Itinerary Assistant). Our application aims to support neurodivergent mobility by finding paths minimizing stress. This stress is computed using a set of variables whose relative importance is tailored for each individual.

Index Terms—Autism Spectrum Disorder (ASD), smart mobility, navigation system, artificial intelligence (AI)

INTRODUCTION

Autism Spectrum Disorder (ASD) affects approximately 1% of the global population, representing nearly 700,000 people in France alone [13].

Individuals with ASD have specific needs in many areas of daily life, particularly regarding their mobility. Certain environmental factors such as noise, crowd density, or route unpredictability can generate significant stress and make daily commutes particularly challenging.

Current navigation applications (Google Maps, Waze, Apple Maps, etc.) primarily optimize routes based on time or distance criteria, without considering the impact of stress, which is critical for people with ASD. This limitation makes these tools poorly suited to their needs and can cause significant anxiety during daily commutes.

To address this issue, we present ASDIA (Autism Spectrum Disorder Itinerary Assistant), an Android navigation application that integrates customizable stress variables into route calculation. Our application enables users to define their specific sensitivities and suggests routes that minimize their exposure to stress while maintaining conventional time and distance criteria.

Our main contributions are threefold.

- 1) First, we identify and model stress variables relevant to people with ASD in the context of urban navigation.
- 2) Second, we develop a routing algorithm that takes these factors into account.
- 3) Third, we design a customizable and accessible user interface.

This paper presents the creation of ASDIA. In Section I, we give an overview of related works in the shape of a state of the art, exploring existing technologies and tools relevant to the presented issue. Section II explains how we integrated stress variables and their measurements to allow our application to provide users with stress-free paths. Section III details the architecture of ASDIA and used technologies. Section IV and V describe the implementation of its two main parts, respectively the client-side graphical and interactive application and the server-side processing and computing algorithm. Finally, Section VI gives an overview of the results of our implemented application.

CONTENTS

Introduction	1
I State of the art	2
I-A Introduction	2
I-B Digital applications for navigation	2
I-C Adapted vehicles and onboard assistance	2
I-D Mixed reality and immersive technologies	2
I-E Adaptive tools and assessment frameworks	3
I-F Conclusion and perspective	3
II Relevant stress variables	3
II-A Stress triggers	3
II-B Personalized Sensitivity Modeling	4
II-C Detailed scenario examples	4
II-D Impact on the application	4
III ASDIA’s architecture and technology choices	5
III-A Client-Server architecture	5
III-B The routing engine	5
III-C The Python API layer	5
III-D The Android client: Kotlin and Retrofit	5
IV Client side: Graphical User Interface (GUI) design and implementation	5
IV-A Visual identity and design principles	5
IV-B Responsive layout architecture	6
IV-C Scenario interface implementation	6
IV-D Map interface implementation	6
IV-E Setting interface implementation	7
IV-F Route selecting interface implementation	7

V	Server side: data fetchers and routing algorithm	8
V-A	Data acquisition and processing pipeline . . .	8
V-A1	System architecture and technology stack	8
V-A2	Pipeline components	8
V-A3	Current limitations and future work	9
V-B	Algorithm and stress optimization	10
V-B1	Graph creation	10
V-B2	Optimized stress path	10
V-B3	Balancing time and stress reduction	11
V-B4	Other stress parameters	11
V-B5	Threaded implementation	11
VI	Results and discussion	11
VI-A	Proof of concept achievements and scalability	11
VI-B	Proactive vs. reactive navigation	11
VI-C	Algorithmic complexity and routing trade-offs	11
VI-D	UI/UX limitations and the need for participatory design	12
	Conclusion and future work	12
	References	12

I. STATE OF THE ART

A. Introduction

Independent mobility is a key factor for social participation and autonomy, yet it remains a major challenge for individuals with Autism Spectrum Disorder (ASD). Sensory sensitivities, difficulties with unpredictability, social interactions, and complex environments can significantly limit access to both public transportation and independent travel. Recent technological advances aim at supporting autistic individuals in their mobility. This paper provides an overview of existing approaches and discusses their contributions and limitations in real-world contexts.

B. Digital applications for navigation

Digital navigation and journey-planning applications are widely explored as technological supports to enhance independent mobility for individuals on the autism spectrum. These tools, often inspired by mainstream navigation systems, aim to provide clearer, more structured, and simplified guidance adapted to the cognitive and sensory specificities associated with ASD. Core functionalities typically include step-by-step navigation, time management through alerts, and the anticipation of key elements along the route.

Several applications also incorporate real-time adaptation capabilities, allowing dynamic route adjustments in response to traffic conditions or service disruptions. By integrating traffic evaluation, they may reduce exposure to crowded environments, while autism-oriented design features—such as pictograms, simplified instructions, and reassurance mechanisms—seek to limit stress and support comprehension. Positive reinforcement strategies are sometimes used to encourage repeated use and confidence in independent travel.

Empirical studies suggest that such applications can improve journey understanding and reduce travel-related anxiety by increasing predictability and perceived control [1]. These

results highlight the potential contribution of digital tools to broader objectives of autonomy and inclusion in mobility.

Nevertheless, important technical limitations remain. Insufficient spatial precision and incomplete or outdated data can affect the reliability of guidance, particularly in dynamic or disrupted transport contexts, which may in turn increase uncertainty and stress for users [1].

Beyond these issues, the literature reveals a gap between application functionalities and the needs expressed by individuals with ASD. User-reported priorities largely concern environmental, social, and organizational aspects of public transport—such as crowd management, rule enforcement, staff support, and system-level accessibility—rather than navigation assistance alone. These needs extend beyond what application-based solutions can address [2].

Overall, while digital applications can support route comprehension and individual coping strategies, they remain insufficient to overcome many structural barriers to public transport use. This limitation points to the relevance of complementary approaches embedded within transport systems themselves, which are examined in the next section.

C. Adapted vehicles and onboard assistance

Adapted vehicles are often considered as a means for people with ASD to gain greater autonomy in their mobility. However, although many people with ASD express interest in driving (e.g., about two-thirds of young adults), and nearly a third already hold a driving licence in the US, driving a car remains a significant challenge. Environmental stimuli, coupled with having to focus on the same task for a long time, and the need to take into account multiple parameters, can easily lead to sensory overload. The fear of an accident alone discourages many autistic individuals from attempting independent travel.

Many individuals rely on a caregiver to drive them safely. Autism-centered carpooling has also been considered. However, the caregiver cannot fully attend to their passenger's needs. Moreover, such arrangements do not provide the level of autonomy sought by individuals with ASD.

A study conducted by the University of Michigan [3] revisited the potential of autonomous vehicles. The authors suggest that a driverless car, combined with a soothing in-vehicle application, as well as an interior designed to be stress-free (e.g., removing the divider, as it might create negative emotions), can support greater autonomy while enhancing passenger comfort.

Unfortunately, several challenges come along with this approach. Developing safe and reliable driverless vehicles is not only a technological hurdle but also restricted in many countries due to legal constraints. Furthermore, such solutions may be largely inaccessible to individuals without financial support. These issues highlight the need for further research on vehicle adaptations specifically for individuals with ASD, a domain that remains largely underexplored.

D. Mixed reality and immersive technologies

Mixed reality (MR), combining augmented and virtual reality, offers significant potential to support the mobility of

people with ASD. By overlaying digital information onto the real environment, it helps reduce uncertainty, structure spatial transitions, and limit sensory overload—difficulties frequently reported during travel [4].

Solutions such as Hol’Autisme, which uses Microsoft HoloLens glasses, demonstrate how MR combined with artificial intelligence can analyze and train sensory-motor and social skills [4]. The proposed modules (e.g., PopBalloons, Smile) help develop coordination, sensory stability, and facial expression recognition—skills indirectly related to independent mobility. However, these tools remain largely confined to supervised therapeutic settings and are rarely evaluated in real transport situations.

The Empowered Brain program, based on Google Glass-type devices, offers functionalities more directly related to travel. Modules such as Transition Master allow users to explore unfamiliar places in advance to reduce anxiety related to transitions, while Paint My World aims to filter the visual environment to limit sensory overload [5]. These approaches illustrate the value of MR in preparing for and regulating mobility experiences, despite technical and social limitations (autonomy, device acceptability).

In addition, guidance systems originally designed for visual impairments—such as Wayfindr, NavCog, NaviLens, or Microsoft Soundscape—show strong potential for adaptation to the needs of people with ASD [6], [7]. They facilitate indoor orientation, management of transfers, and access to contextual information, helping to secure travel [8]. Tools like XRAI Glass, which displays audio announcements in real time as text, also address auditory comprehension difficulties that are common in transport environments [9].

Despite these advances, the use of MR for the daily mobility of adults with ASD remains limited. The main barriers include the cost of devices, cognitive fatigue, a lack of validation in real-world conditions, and ethical issues related to data use [10]. Nevertheless, the future integration of MR with mobile applications and intelligent transport systems represents a promising pathway toward truly inclusive mobility.

E. Adaptive tools and assessment frameworks

People with Autism Spectrum Disorder experience difficulties that may affect communication, social interactions, or sensory processing. However, symptoms can vary considerably from one individual to another. As a result, adaptive tools represent a promising approach, making it possible to design services that respond to the specific needs of each autistic person in order to promote their autonomy, comfort, and inclusion. These tools are therefore mainly analytical tools that help identify the characteristics of an individual’s autism.

For example, the TSAT (Transportation Skills Assessment Tool) is a tool which is still under development. It consists of a test made up of 37 tasks aimed at observing a person’s cognitive and practical abilities in transportation-related situations. Although the tool appears to be functional, the pilot study involved only a very small number of participants. Therefore, the results are preliminary and cannot be generalized to all

adults with autism. Moreover, the TSAT does not create transportation services, it merely assesses access to existing ones. If an adapted transportation network does not exist in the geographical area, the tool remains of limited interest. [11]

Another interesting tool is Mind the Gap. This tool aims to assess the accessibility of public transportation for autistic people. The objective is to identify sensory, organizational, or social barriers in order to propose better-adapted spaces, services, and products. However, this tool was developed as part of a thesis and is not (yet) a widely operational or generalized service. The tool exists, but it has not necessarily been implemented within transportation networks. [12]

Thus, TSAT and Mind the Gap are solutions that lack experience in “real-life” situations. Moreover, taken individually, these tools are not sufficient to transform the mobility experience of autistic people. However, when combined, they offer a more comprehensive perspective: one evaluates individual skills, while the other analyzes the environment. Together, they pave the way for truly inclusive services.

F. Conclusion and perspective

This literature review examined existing approaches to support the mobility of individuals with Autism Spectrum Disorder, including digital navigation applications, adapted vehicles, mixed reality technologies, and adaptive assessment tools. These solutions aim to reduce uncertainty, sensory overload, and anxiety, while fostering greater autonomy in daily travel.

The investigation shows that while digital and immersive tools can improve route comprehension and preparation, they remain limited by technical constraints. Similarly, autonomous vehicles present promising opportunities but face significant technological, legal, and economic challenges. Furthermore, these solutions focus mostly on individual-level adaptation.

Despite these limitations, applications and mixed reality seem to be the most promising approaches towards inclusive mobility.

II. RELEVANT STRESS VARIABLES

In order to propose an application aiming at computing low stress routing, the first challenge we encountered was to rigorously define and measure stress.

We first proceeded to search for a set of stress variables relevant to the needs of individuals with ASD (see Section II-A) and to a routing application. We then created a form in the shape of scenarios (see Section II-B) to understand the sensitivity of the users to each of these variables.

A. Stress triggers

To define the stress-related variables integrated into ASDIA, we first conducted a preliminary analysis of common environmental triggers reported by individuals with Autism Spectrum Disorder. These triggers were:

- Noise sensitivity: loud sounds can be disturbing and raise anxiety whereas cumulative sounds such as conversations can lead to auditory overload.

- **Light sensitivity:** numerous intense lights can lead to sensory overload, while the absence of light can also be stressful due to the reduced ability to rely on vision.
- **Crowd density:** crowd density is easily one of the most decisive triggers for individuals with ASD because it can create a lot of different sensory overloads at once (sounds, movements, physical contacts, conversations, etc.)
- **Physical contact:** physical contacts can be a source of stress, especially if not anticipated. In practice we were unable to directly measure this trigger and considered that physical contact was proportional to crowd density.
- **Unpredictability:** last minute changes, delays, and the need for individuals to make decisions under time pressure can lead to overwhelming anxiety. Changes in routine habits, for example due to roadworks, can also trigger unease.

Unfortunately, some other existing triggers, such as smell sensitivity, are close to impossible to integrate due to the difficulty of finding corresponding data. Consequently, we did not include these factors in our application, despite their potential importance.

B. Personalized Sensitivity Modeling

To allow the application to give results tailored to the needs of each user, we need to give each of these stress variables personalized scores. This way, we are able to take into account which sensitivities are most significant for each individual, thus returning a path focusing on avoiding roads that are likely to be stressful for the user.

However, traditional data collection methods, such as standard questionnaires, rely on abstract concepts. They ask users to rate their feelings on a scale of one to ten or answer hypothetical questions like "Do you find crowds stressful?". This requires high-level abstraction and self-reflection, which can be cognitively demanding and potentially anxiety-inducing for individuals with ASD. Such methods often fail to capture the visceral, real-time nature of stress responses.

Instead of relying on text-based surveys, we designed a set of interactive and immersive scenarios that simulate real-life navigation situations. These scenarios combine audio, video, images, and short text-based choices to capture users' reactions in a more intuitive and engaging way.

The core design principle is progression. Each scenario begins with a low-intensity stimulus and gradually increases in complexity and stress. Crucially, every scenario can be interrupted at any time by the user. This "stop point" allows users to indicate precisely when discomfort or stress arises, providing a precise measurement of their individual tolerance thresholds. This approach transforms data collection from a passive test into an active, user-centered experience, making the onboarding process faster, more accessible, and less cognitively demanding.

C. Detailed scenario examples

In this section we describe how the scenarios were designed and their characteristics.

- **Noise sensitivity:** Noise-related scenarios are designed to evaluate the participant's sensitivity to auditory stimulation in urban transportation environments. These scenarios expose participants to different types of sound conditions, including progressively increasing ambient noise and isolated loud events. Participants can interrupt the experience at any time or report their discomfort level using rating scales. The objective is to identify both tolerance thresholds to cumulative auditory stimulation and reactions to sudden intense sounds.
- **Light sensitivity:** Light-related scenarios assess visual sensitivity to different lighting conditions commonly encountered in urban settings. Participants are exposed to environments with varying levels of brightness, reflections, and dynamic light sources such as headlights or flashing signs. Additional preference-based tasks explore which visual environments are perceived as comfortable or stressful. These scenarios aim to characterize discomfort caused by intense, flickering, or highly dynamic visual stimulation.
- **Crowd and physical proximity:** Crowd-related scenarios investigate the participant's tolerance to dense public environments and physical proximity with strangers. Simulated transportation situations progressively increase social density and reduce personal space. Participants indicate the moment at which discomfort or stress appears and answer follow-up questions regarding their willingness to remain in such situations despite inconvenience or delays. These scenarios help evaluate stress responses associated with crowded mobility contexts.
- **Unpredictability and decision-making:** Unpredictability scenarios explore how participants react to uncertainty, delays, and real-time changes during transportation. Through text-based decision tasks, participants express their preferences regarding waiting tolerance, route modifications, predictability, and trade-offs between stress reduction and travel efficiency. These scenarios aim to measure behavioral flexibility and the importance of control and predictability in navigation decisions.

D. Impact on the application

By transforming data collection into a gamified and user-centered experience, we ensure that users remain engaged while efficiently providing meaningful insights into their individual sensitivities. The "stop points" and choices are converted into numerical scores, creating a unique stress profile for each user. This profile is not theoretical but based on actual behavioral reactions. These personalized constraints are then fed directly into the routing algorithm, allowing ASDIA to compute a path that minimizes exposure to the specific triggers tailored to each user, ensuring a truly safe and comfortable navigation experience.

III. ASDIA'S ARCHITECTURE AND TECHNOLOGY CHOICES

A. Client-Server architecture

During the design phase, the first challenge we encountered was how to make the app efficient and fast. It became clear that performing route computation directly on the user's smartphone would not be viable in the long term and would be hard to maintain and keep updated. The implementation relies on a path-finding algorithm which could become computationally expensive on a local device. More importantly, the roadmap for ASDIA includes integrating AI such as a machine learning model to improve personalization based on users' behaviour and environmental data. Running such a model on a smartphone, even a recent one, would result in significant battery drain, longer response times, and device overheating, all of which would degrade users' experience.

Therefore, a client-server model was adopted: the heavy computation is offloaded to a dedicated server, while the mobile application remains lightweight and responsive, and will only request and display the computed results. This separation also makes it straightforward to update or replace the routing engine, trying other algorithms or adding new features easily in the future without requiring a new release of the Android application.

This implementation does not compromise users' personal information since the requests sent from the client-side application are anonymous.

B. The routing engine

The core routing engine is written in C++. To ensure minimal latency for route generation, this choice was driven by performance considerations: C++ offers low-level memory control and execution speed that are well suited to computationally intensive graph traversal algorithms. The engine can then ensure that the computation pipeline runs efficiently without the overhead that would come with a higher-level language. Communication between the Python API layer (see section III-C) and the C++ engine is handled via ZeroMQ (ZMQ), a lightweight messaging library that allows the two processes to exchange JSON payloads over a local TCP socket with minimal latency. See section V for further details.

C. The Python API layer

Sitting between the Android client and the C++ engine is a REST API written in Python using FastAPI. This intermediary layer handles request validation (via Pydantic models), logging, and error handling before forwarding requests to the C++ engine and returning a structured response to the client. FastAPI was chosen for its simplicity, its automatic data validation, and its native support for asynchronous request handling, which enables the server to remain responsive while waiting for the engine to return a result. This Python API layer provides a clean, maintainable interface between the engine and any client. Section V explains the details of this implementation.

D. The Android client: Kotlin and Retrofit

The Android application was developed in Kotlin, the modern standard for Android development. This application serves as the client side of our architecture.

HTTP communication was handled with Retrofit, an HTTP client library for Android. Retrofit ensures safe network calls to be defined as simple Kotlin interfaces, keeping the networking isolated from the rest of the application. Each request to the server is defined as a 'suspend' function, allowing it to be called within a Kotlin coroutine without blocking the main thread.

The data exchanged with the server is serialized and deserialized using JSON for forwarding them. A DataSending object contains the departure coordinate, the destination coordinate, and the user's sensory profile. It is automatically converted to JSON by Retrofit before being sent to the server's '/api/v1/compute_route' endpoint. The server's response, which includes one or more candidate routes with their coordinates, estimated travel time, distance, and stress score, is then deserialized back into Kotlin data classes for the application to display.

The sensory profile captures parameters such as noise sensitivity, crowd sensitivity, need for seating, and aversion to flashing lights all on a numerical scale. They are included with every route request so the server can tailor its suggestions accordingly. This data is then returned to the app which displays the optimal path to the user.

All of this ensures that this infrastructure is accessible from a mobile application in a way that is both performant and easy to maintain.

The implementation details are explained below.

IV. CLIENT SIDE: GRAPHICAL USER INTERFACE (GUI) DESIGN AND IMPLEMENTATION

The graphical interface of ASDIA was designed with two primary objectives: ensuring accessibility for autistic users and providing a clean, modern, and responsive experience across all Android devices.

The interface is not merely a visual layer; it plays a central role in the onboarding process, as it is the medium through which users interact with the stress-assessment scenarios and visualize their returned path. Consequently, its design required a combination of ergonomic considerations, aesthetic coherence, and technical optimization.

A. Visual identity and design principles

The visual identity of ASDIA was developed to create a calm, reassuring, and non-overstimulating environment. The color palette is intentionally composed of soft greens and desaturated tones, selected to create a calmer and less visually overstimulating interface. High-contrast elements (white text on dark backgrounds) ensure readability without introducing aggressive visual stimuli.

Our logo features a brain-shaped icon crossed by a pathway, symbolizing cognitive comfort and the app's ease of use.

All scenario pages follow a unified layout structure to reduce cognitive load: a header, a central content block (video, image, or question), and a bottom navigation bar (see Fig. 1). This consistency helps users anticipate interactions and reduces stress caused by unpredictable layouts.

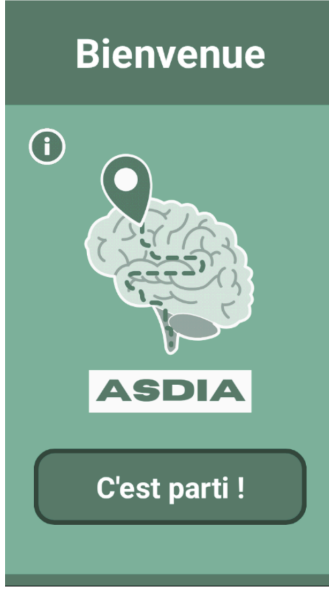


Fig. 1. Starting page

B. Responsive layout architecture

To ensure compatibility across a wide range of Android devices (from small smartphones to large tablets), the interface was built using `ConstraintLayout`, which enables fine-grained control over alignment, proportional sizing, and adaptive positioning.

Used key techniques include:

- **Percentage-based constraints:** Headers, question blocks, navigation bars or other containers use `layout_constraintHeight_percent` and `layout_constraintWidth_percent` to scale proportionally with screen size. This ensures that the structure of each page remains stable regardless of resolution.
- **Auto-sizing text:** Text elements use Android's `autoSizeTextType="uniform"` to dynamically adjust font size based on available space. This prevents overflow on small screens and avoids excessively large text on tablets. To guarantee predictable behavior, auto-sizing is applied only when the `TextView` text width is constrained (e.g., `layout_width="0dp"` with start/end constraints).
- **Avoidance of deprecated or unstable layouts:** A lot of legacy layouts such as `FrameLayout` or `GridLayout` were replaced by nested `ConstraintLayout` blocks to ensure predictable behavior. This was particularly

important for aligning text labels around interactive elements (e.g., `SeekBar`s), where precise left/right anchoring was required.

C. Scenario interface implementation

As detailed in section II-B, each scenario page integrates multimedia elements (audio, video, images) and interactive components (buttons, sliders) (see Fig. 2). The implementation requires several technical considerations:

- **Video integration:** Videos are embedded using `VideoView` with a fixed aspect ratio ('16:9') enforced through `ConstraintLayout`'s `DimensionRatio`. This ensures that videos remain fully visible without cropping or distortion.
- **Stress-response capture:** User responses are collected through:
 - Multiple-choice buttons, each styled with custom drawable backgrounds to visually represent stress levels.
 - `SeekBar`s, used for fine-grained stress intensity selection.

Labels are positioned using `ConstraintLayout` anchors to ensure perfect left/right alignment.

- **Navigation system:** A bottom navigation bar provides "previous" and "next" buttons, along with a progress indicator. This bar is proportionally sized and remains fixed at the bottom of the screen, ensuring consistent navigation across all scenarios.



Fig. 2. Scenario-based questionnaire interface

D. Map interface implementation

The map is the core functionality of the application, where users can select their destination (and starting point, if necessary) and visualize the route generated by the algorithm. In order to take into account the sensitivity of individuals with ASD to unpredictability and changes (see section II-A), we added the ability to save routes as favorites. Users can easily

reuse these routes in the future, as following a familiar and routine path for recurring journeys can provide reassurance.

The Map page (see Fig. 3) uses maps from the open street map API. An airy interface aims to prevent users' overstimulation by limiting the number of available functionalities. Users can either access the settings, center the map on their current location, orient the map with north at the top, and of course, an access to the route selecting interface (described in section IV-F).

Whenever a new route has been computed, the interface changes and two new green buttons appear: one enabling users to center the map on the destination and the other allowing to add the route to their favorites.

The color blue was chosen to indicate the routes, as it contrasts with the green tones of the application without being overwhelming and making the routes easier to distinguish on the map.

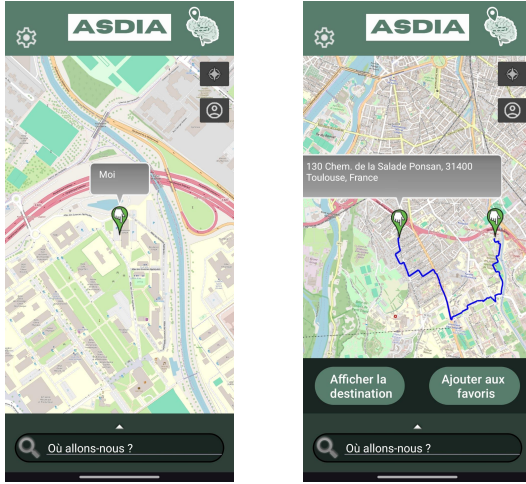


Fig. 3. Main map interface

E. Setting interface implementation

The page dedicated to the settings is intentionally minimalist, aiming to prevent users' overstimulation, displaying different categories of information with buttons (see Fig. 4).

Key feature of the implementation include updating personal data, managing bookmarked routes, adjusting parameters and displaying information.

1) Updating personal data

a) *User Interface:* This interface contains a large amount of information, which conflicts with our goal of keeping the interface as uncluttered as possible. However, it was designed to provide users with detailed control over the algorithm's parameters. In particular, the different scores given to the stress-variables computed through the scenarios (see Section II-B, can be modified according to the users' will through sliders that define acceptable stress levels (see Fig. 5).

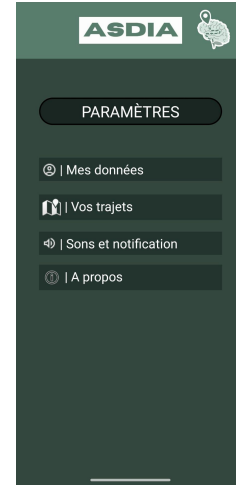


Fig. 4. Settings page

b) *Data management:* All users' data collected with the questionnaire are only kept inside the users' devices and have the unique purpose of serving in the path-finding algorithm. The data received by the server are anonymous.

2) Management of bookmarked paths

One of the most important features is the ability to save up to five favorite routes, which can be accessed even while being offline (see Fig. 5) since they do not need to be recomputed. This implementation was primarily motivated by the variability of generated routes and the need to accommodate individual user preferences, particularly the desire to maintain consistent routines.

3) Adjusting the App parameters

As in any application, users can easily modify, sound volumes, brightness, font sizes and others to match their preferences. Note however that the level of customization is low, as this would be an unnecessary overload.

4) Displaying App information

The information interface provides important details about the application, including its purpose, contextual information, and notices regarding future updates.

F. Route selecting interface implementation

The interface for selecting route contains only three fields for entering the departure address, arrival address, and desired arrival time (see Fig. 6).

These fields use Android's native Geocoder to predict and suggest addresses based on the users' input. Currently, we assume the users want to leave immediately, therefore there are no dedicated fields to enter the departure time. (This decision is based on the fact that route calculation can fluctuate depending on environmental factors.).

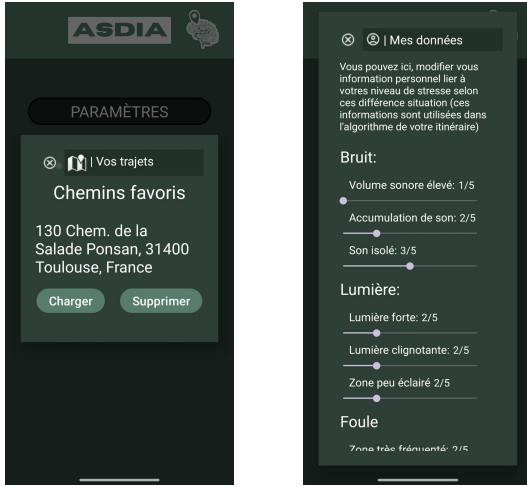


Fig. 5. User settings customization and path-saving functionalities

Once the necessary fields have been filled by a user, a request is sent to the server. Upon reception of the result, the application shows the map interface with the resulting route displayed.

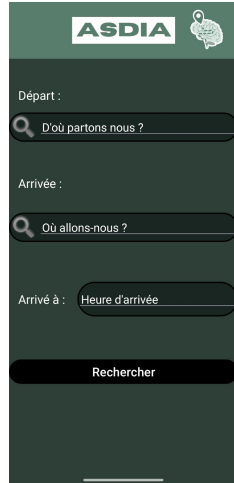


Fig. 6. Destination search interface

V. SERVER SIDE: DATA FETCHERS AND ROUTING ALGORITHM

Once the application sends a routing request, it is received and processed by the server.

A. Data acquisition and processing pipeline

The primary objective of our application is to aggregate environmental data in order to compute a “stress score” for different geographical routes. The routing algorithm can

then suggest a stress-minimizing route for users. A periodic background service is responsible for fetching, processing, and formatting spatial data.

1) *System architecture and technology stack:* Like other computationally intensive tasks, the data retrieval pipeline is executed entirely server-side, as running periodic data-fetching tasks on every client would be impractical and inefficient. The backend service is developed in Python. Although languages like C++ offer superior raw performance, Python was chosen for its rapid development capabilities, its extensive ecosystem for HTTP requests, and its powerful geospatial libraries (such as Geopandas). Since this pipeline operates as an asynchronous background task, ultra-low latency is not a strict requirement. The architecture is highly modular to facilitate future additions and is divided into four main components:

- Main Pipeline
- Fetchers
- Normalizer
- Geo Matcher

2) Pipeline components:

1) Main pipeline

The Main pipeline acts as the orchestrator. Its most computationally intensive task is reading the OpenStreetMap (OSM) base map [15]. We use the Protocolbuffer Binary Format (PBF), which is optimized for compact size and fast read performance. The file is parsed using PyOsmium [16], a Python wrapper for the C++ libosmium [17] library. This approach enables high-level Python logic while benefiting from the memory efficiency of the underlying C++ implementation. To minimize RAM usage and computational time, the PBF file must be read only once. To achieve this, we implemented a custom MultiOSMHandler that aggregates multiple simulated data handlers. As the parser iterates through the map, it triggers the respective methods for every handler simultaneously when encountering a “node” (a specific coordinate) or a “way” (a path or polygon):

```
import osmium

class MultiOSMHandler(osmium.SimpleHandler):
    def __init__(self, handlers):
        super().__init__()
        self.handlers = handlers # List of
                                # simulated data handlers

    def node(self, n):
        for h in self.handlers:
            if hasattr(h, "node"):
                h.node(n)

    def way(self, w):
        for h in self.handlers:
            if hasattr(h, "way"):
                h.way(w)
```

This single-pass reading is crucial for performance and ensures the system remains scalable if real-time data handlers are added in the future. In the absence of specific external data, this step also infers baseline simulated data directly from OSM tags.

2) Data fetchers

The second component is composed of HTTP clients that retrieve external datasets. We currently rely on two main providers:

- **Noise-Planet:** A CNRS project mapping decibel levels via crowdsourced measurements [18].
- **Data.gouv.fr:** The French government’s open data portal [19].

Initially, the fetchers used paginated API calls. However, to prevent overloading the providers’ free APIs, the architecture was refactored to download bulk data files and leverage HTTP ETag header validation. This acts as a version identifier, allowing the system to skip the download process entirely if the local file is already up to date with the server.

3) Normalization and stress scoring

Once both simulated and fetched data is collected, they pass through the Normalizer. This module standardizes diverse data formats into a unified GeoDataFrame (using the geopandas library), which associates tabular data with spatial geometries (points, lines, or polygons). The standardized schema contains three main columns: `geometry`, `factor_type` (e.g., `autism:noise_level`), and `score`.

The `score` represents the environmental stress level normalized on a scale from 0 (minimum stress) to 100 (maximum stress). Assigning these scores involves both direct mapping and heuristic modeling:

- **Direct mapping:** For noise levels, the decibel value is directly mapped to the score (e.g., 65 dB equals a score of 65/100).
- **Binary scoring:** For areas with roadworks or constructions, related to the unpredictability stress-variable (see section II-A), due to the complexity of evaluating specific site impacts, a binary approach is currently used: 0 for no construction, and 100 if construction is present.
- **Heuristic modeling:** Some environmental factors, particularly crowd density and sudden noise sources, cannot be inferred reliably from freely available data sources. Their associated stress levels are therefore approximated using contextual indicators such as time of day, day of the week, and proximity to high-activity areas including schools, hospitals, marketplaces, and shopping centers. Spatial heuristics are also used to approximate sensory impact: for example, airports are assigned a score of 100 within a 1 km radius, fire stations a score of 90 within 250 meters, while restaurants and parking lots receive scores of 30 within radii of 10 and 25 meters,

respectively.

4) Geo matcher and output generation

The final processing step requires a spatial intersection between the OSM map’s “ways” and the normalized stress factors. This is efficiently executed using Geopandas’ spatial join function:

```
joined = gpd.sjoin(osm_roads, factors_gdf,
                  predicate="intersects", how="inner")
```

Determining the final score for a path segment intersecting multiple stress zones is a critical design challenge. We opted for a “pessimistic routing” approach: the final score of a segment is the maximum score among all intersecting factors. This approach prevents the routing algorithm from exposing users to unexpectedly high-stress environments.

Finally, the processed data is exported to a CSV file. This format was chosen because it can be parsed extremely quickly by the C++ routing engine. Each row contains the `way_id` and the corresponding scores for each stress factor. An example of a representative output file is shown in Table I.

TABLE I
EXAMPLE OF THE PIPELINE OUTPUT FILE. EACH ROW REPRESENTS AN OSM WAY; EACH COLUMN REPRESENTS A NORMALIZED STRESS FACTOR SCORE (0–100).

way_id	construction	crowd	lackofflight	noise_stress	noise_db
1863458	0.0	50.0	0.0	100.0	0.0
2304962	0.0	50.0	100.0	80.0	59.8
2304963	0.0	50.0	0.0	80.0	0.0

3) *Current limitations and future work:* While functional, the current data retrieval and scoring pipeline has several limitations that we aim to address in future iterations:

- 1) **Lack of real-time data:** Our application currently cannot account for temporary events (e.g., a localized festival or sudden crowd gathering). Unlike large tech companies that leverage millions of connected devices to estimate real-time crowd density, we do not have access to such dynamic data through open APIs. Some proprietary APIs appear to provide this type of service, but their reliability and responsiveness could not be evaluated due to access and cost constraints.
- 2) **Data reliability and sparsity:** Some retrieved data are unreliable or sparse, forcing us to generalize over large areas, such as regions lacking street lighting or current noise levels. A proposed solution is to implement an optional opt-in crowdsourcing feature within the app. This would allow users to report environmental conditions, thereby refining the map’s accuracy and contributing back to the open-source community.
- 3) **Arbitrary scoring models:** A number of scoring decisions rely on rule-based approximations and may not fully reflect the lived experience of autistic individuals. This includes the binary scoring of construction

works, the worst-case aggregation strategy, and the spatial extents and score values assigned by the amenity-based heuristic models. Calibrating these parameters in a principled manner would require empirical data on the sensory responses of autistic individuals to specific environmental stimuli, data that is difficult to obtain given the high variability of autism spectrum presentations and the ethical constraints surrounding such studies. This represents a significant avenue for future work, ideally in collaboration with specialists in autism research and with the autistic community itself.

B. Algorithm and stress optimization

Once the stress-enriched map is obtained, an algorithm can compute least stressful paths to answer clients' requests. In order to execute a path-finding algorithm, a graph is needed. Thus the stress enriched map is processed into a stress enriched graph through a method described in V-B1. The computation of the stress-minimizing path is then explained in V-B2 and the balancing between time and stress optimization is related in V-B3. Furthermore, stress parameters than cannot be minimized are discussed in V-B4. Finally, answering the risk of high requests inflow is achieved using a multi-threaded implementation portrayed in V-B5.

1) *Graph creation*: The graph's implementation is decisive to gain time during the execution of the algorithm. For a quick execution time, not creating the graph each time the program is called (see section V-D5) allows us to optimize graph reading, this even if the creation stage is noticeably longer.

The graph is implemented using adjacency lists (to each node is associated a vector containing all its neighbour nodes and edges). A given Edge contains the distance between its two nodes, the time necessary for it to be crossed, and the Stress object associated to it. This implementation is the most effective for the algorithm's implementation (see section V-B2). Accessing time and stress levels is done with constant complexity, and do not take much memory.

Creating the graph is done through reading all stress parameters from the Python program (see 3). The stress variables are saved in a data structure used during the graph's creation. All the variables are linked to an ID that represents a specific edge in the Open Street Map file.

Once all stress variables are read, the graph starts being build. In an Open Street Map file, a considerable part of the nodes are unnecessary. For example, the file contains nodes indicating the presence of buildings or trees. To find the user's path, we only need information over the roads accessible for pedestrians. We refer to them as "allowed nodes". To keep only allowed nodes, the file is read two times.

During the first reading, the nodes unreachable by pedestrians are filtered out.

However, in one sole street, in which changing direction isn't possible, many unnecessary nodes are kept. To gain performance during the execution of the algorithm by visiting fewer nodes, the generated graph should contain only the intersections. In order to achieve this result, the file is read

a second time. The nodes common to multiple roads are naturally the intersections. Stress, computed on the stress enriched map (see section 3), is also reported to the graph during this section.

2) *Optimized stress path*: Given a user's request an origin node O , a destination node D and the list of the user's personal stress variables valuations $V = [v_1, \dots, v_K]$ where K is the number of stress variables. These stress variables correspond to sound sensitivity, light sensitivity and crowd sensitivity.

The stress enriched graph is a graph $G = (N, E)$ where $N = \{n_i | i \in [0, n]\}$ is the set of the nodes and $E = \{e_j | j \in [0, m]\}$ the set of the edges. Each edge e_j possesses a list $S_j = [s_{j1}, \dots, s_{jK}]$ of stress values for each stress variable.

The stress s_j value of an edge is the mean value of its stress list weighted by the user's valuations.

$$s_j = \frac{1}{K} \sum_{i=1}^K v_i \times s_{ji}$$

Our aim is to find a path from O to D minimizing the mean of all the used edges stress' values. If $P = [e_1, \dots, e_p]$ is a path from O to D , its mean stress value s_P is

$$s_P = \frac{1}{p} \sum_{e_j \in P} s_j$$

Example: if $P_{ex} = O \xrightarrow{e_1} n_1 \xrightarrow{e_2} n_2 \xrightarrow{e_3} n_3 \xrightarrow{e_4} D$ (where, $e_1, e_2, e_3, e_4 \in E$ and $n_1, n_2, n_3 \in N$),

$$s_{P_{ex}} = \frac{1}{4}(s_a + s_b + s_c + s_d)$$

The most commonly used algorithm for finding minimal paths in graphs is Dijkstra's algorithm.

Dijkstra's algorithm performs a breadth-first search starting from the Origin point, when visiting a node it maps its neighbours distances to the origin. If a distance was already found it only updates it if the new distance is shorter than the previous path to that neighbour node. The next node to be visited is an unvisited node which distance from the origin is the shortest among all. The algorithm finishes when the destination node is visited or once all reachable nodes have been visited (in which case there is no path linking the origin and destination).

Due to mathematical reasons, when working with positive costs and when the cost of a path is the sum of its edges costs, once a node is visited this means it has reached its final, minimal distance from the origin. Thus, when visiting the destination we have found a path that gives the shortest distance.

However, the cost of a path we aim to minimize is the mean value of its edges costs (stress values) and not their sum. This prevents Dijkstra's algorithm from correctly functioning. A node doesn't reach its strictly minimal cost (eg. stress) value when it is visited. Thus a node could have its minimal cost updated after it has been visited which could result in the creation of cyclic paths (eg. loops) making it impossible to retrieve any route.

To answer this issue, we chose to prevent the algorithm from modifying the distance to an already visited node even if the new cost (eg. stress) is smaller. Ultimately, the algorithm does return a path which is not strictly optimal. It is however a very good approximation of a minimal stress path and benefits from the fast computation of Dijkstra’s algorithm.

We also considered that finding the true least stressful path was not decisive because of the consideration of time. Essentially, finding a path reducing purely stress has limitations. Users may not be interested in a low stress path dozens of times longer than the shortest path.

Thus, we had to choose a balance between time and stress minimization.

3) *Balancing time and stress reduction*: To find a good balance for a path to not be too long while focusing on optimizing stress cost we considered various options. We first contemplated returning multiple paths to the user, they would then choose in a buffet like manner the path they prefer between paths more time reducing oriented and paths more stress reducing oriented. There would have been most likely three paths for the user to choose from. However, we judged this method to not answer the demand since choice overload could be detrimental to individuals with ASD. Furthermore, computing these different paths would already be challenging.

The solution we adopted is a “time margin”. The user chooses a time margin between 1 and 3 with a default value computed thanks to their personal data from the application’s questionnaire.

We then run a purely time minimizing algorithm using Dijkstra’s algorithm (fully optimal in this case, and guided with speed heuristics (note that heuristically guided Dijkstra’s algorithm is generally called A*)) to find the time of the shortest path to exist between the origin and destination.

Finally the least-stressful algorithm describes above in section V-B2 checks before updating the stress cost of a neighbour node if the cost in time (eg. time to reach it from the origin) is inferior to:

$$\text{time margin} \times \text{shortest path's time}$$

If it is superior to this value, the node is not examined.

This ensures that the time of the solution returned by the stress minimizing algorithm is at most the time margin $\times$ the shortest path’s time.

Furthermore, this reduces the overall exploration of the algorithm to the subset of paths feasible in the limited time. Thus speeding up the computation.

4) *Other stress parameters*: We translated the unpredictability variable (described in section II-A) as a need to avoid roadworks. A lack of real-time data prevented us from considering other data, such as transport’s delays. This variable is not taken into account in the mean stress value since it has a boolean value, true if roadworks should be avoided and false if not.

To account for this variable when the user’s valuation is that roadworks should be avoided, we simply forbid the algorithm’s exploration to use edges on which roadworks are happening.

5) *Threaded implementation*: Finally, to ensure that the algorithm is able to support a consistent inflow of requests, we chose to make multiple worker threads. When a request is intercepted by the server, it is added to a request’s queue.

Threads that are not already running the algorithm are continuously looking into the queue to check the arrival of the requests. The fastest of the worker threads to detect the new request immediately locks the queue to prevent others to take it. It then takes the request, frees the queue and proceeds to run the algorithm on the request’s parameters.

If more than the number of working threads requests arrive at once, they will simply wait in the queue for a thread to finish its current task and take it.

This system also allows worker threads to run graph update requests.

VI. RESULTS AND DISCUSSION

A. Proof of concept achievements and scalability

Ultimately, this project successfully delivered a comprehensive, end-to-end Proof of Concept (POC), encompassing everything from the initial user-profiling questionnaire to the intelligent navigation system. While the current iteration possesses practical limitations that must be addressed in future development, it successfully demonstrates the viability of the concept. This functional prototype, developed entirely using open-source tools by a team of engineering students working concurrently with their academic studies, serves as a strong indicator of the project’s scalability. With dedicated funding and a full-time development team, this technology possesses considerable potential to be scaled up, highlighting the importance of investing in digital accessibility tools to foster the social integration of individuals with ASD.

B. Proactive vs. reactive navigation

Currently, there are other navigation applications on the market that attempt to assist autistic individuals by offering alternative routes to avoid crowds or noisy environments, such as ClearPath [20]. However, a key difference of our application lies in its routing philosophy. Existing solutions typically adopt a reactive approach, warning the user of upcoming disruptive zones so they can prepare. In contrast, our application focuses on a proactive approach, algorithmically prioritizing the complete avoidance of these sensory triggers whenever possible.

C. Algorithmic complexity and routing trade-offs

While this proactive avoidance is the core strength of our solution, it introduces a major technical challenge. Computing a strictly optimal path on multiple parameters (each stress variable) is a challenging problem, requiring significant computational time. Therefore, as detailed in Section V-B, we chose to minimize means on all variables. This resulted in an algorithm that is not strictly optimal, but provides a good approximation while maintaining fast computation times.

However, a significant trade-off remains: strictly prioritizing a “stress-free” route can sometimes result in detours extending the journey by several kilometers. Currently, this issue is

exacerbated by our data retrieval methods and our stress-scale estimations. Due to a lack of clinical data, our algorithm defaults to a "worst-case scenario" assumption for sensory triggers. We addressed this issue by adding a time constraint, computed by a first algorithm described in section V-B3, which restricts the search space to feasible trajectories.

Future iterations could resolve this by integrating Machine Learning algorithms to dynamically weigh routing options. This would not only optimize the path-finding but also mitigate the limitations previously discussed in the fetchers section.

D. UI/UX limitations and the need for participatory design

Another critical area for future evaluation is the Graphical User Interface (GUI). Our design choices were guided by established accessibility heuristics aimed at reducing cognitive load, such as minimizing color palettes, reducing the number of interactive buttons, and utilizing highly legible icons. However, because our development team does not include any individual with ASD, it is difficult to accurately predict the real-world usability of the app. This is particularly relevant given the vast heterogeneity of the autism spectrum.

CONCLUSION AND FUTURE WORK

Developing this system required making several assumptions regarding both technical implementation and user psychology. To validate these choices and refine the application, future development must transition to a multidisciplinary approach. Collaborating directly with specialized healthcare professionals, occupational therapists, and, most importantly, conducting participatory co-design sessions with users with ASD, will be essential to ensure the application truly meets the complex needs of its target demographic.

REFERENCES

- [1] Kramer, J. M., Kramer, J. C., Howe, T. H., & Bennett, F. C., "Evaluating the effectiveness of an autism-specific public transport app for individuals on the autism spectrum: A pilot study." *Journal of Autism and Developmental Disorders*, 2020.
- [2] Falkmer, T., Andersen, M., Joosten, A., & Falkmer, M., "Enhancing the capacity of individuals with autism spectrum disorder to use public transport." *Travel Behaviour and Society*, 2015.
- [3] GANDHIMATHI PADMANABAN, NATHANIEL JACHIM, HALA SHANDI, LILIT AVETISYAN, GARRETT SMITH, HOWRAA HAMMOUD, FENG ZHOU, "An Autonomous Driving System - Dedicated Vehicle for People with ASD and their Caregivers", University of Michigan, Dearborn, USA
- [4] M. Leharanger, E. A. Rodriguez Martinez, O. Balédent, and L. Vandromme, "Familiarization with Mixed Reality for Individuals with Autism Spectrum Disorder: An Eye Tracking Study," *Sensors*, vol. 23, no. 14, Art. no. 6304, July 2023.
- [5] N. T. Sahin, N. U. Keshav, J. P. Salisbury, and A. Vahabzadeh, "Second Version of Google Glass as a Wearable Socio-Affective Aid: Positive School Desirability, High Usability, and Theoretical Framework in a Sample of Children with Autism," *JMIR Human Factors*, vol. 5, no. 1, p. e1, Jan. 2018.
- [6] Y. Zhao, E. Kupferstein, H. Rojnirun, L. Findlater, and S. Azenkot, "The Effectiveness of Visual and Audio Wayfinding Guidance on Smartglasses for People with Low Vision," in *Proc. 2020 CHI Conf. Hum. Factors Comput. Syst.*, Apr. 2020, pp. 1–14.
- [7] D. Ahmetovic, C. Gleason, C. Ruan, K. Kitani, H. Takagi, and C. Asakawa, "NavCog: a navigational cognitive assistant for the blind," in *Proc. 18th Int. Conf. Hum.-Comput. Interact. Mobile Devices Serv.*, Sept. 2016, pp. 90–99.
- [8] K. Krasowska and A. Zwoliński, "Accessible city – using digital technologies to improve the accessibility of public space for persons with specific needs," *Space & Form*, no. 52, 2022.
- [9] Xrai, "Xrai Glass – Mixed reality for accessibility," [Online]. Available: <https://xrai.glass/>
- [10] M. H. Abidi, A. N. Siddiquee, H. Alkhalefah, and V. Srivastava, "A comprehensive review of navigation systems for visually impaired individuals," *Heliyon*, vol. 10, no. 11, p. e31825, June 2024.
- [11] P. Szary, C. Feeley, "Development of a Transportation Skills Assessment Tool (TSAT) for Individuals with Autistic Spectrum Disorder to Aid in Finding Safe and Accessible Paratransit Service", June 2012.
- [12] G. Musumeci, "Mind the gap: a toolkit to evaluate autism accessibility to public transport services", dec. 2023.
- [13] Inserm. <https://www.inserm.fr/dossier/autisme/>
- [14] Axopen. <https://www.axopen.com/blog/2021/01/retrofit-projet-android-kotlin/>
- [15] OpenStreetMap. <https://www.openstreetmap.org/>
- [16] PyOsmium. <https://osmcode.org/pyosmium/>
- [17] libosmium. <https://osmcode.org/libosmium/>
- [18] Noise-Planet. <https://noise-planet.org/>
- [19] Data.gouv.fr. <https://www.data.gouv.fr/>
- [20] ClearPath. <https://www.manvitaboyini.com/clearpath>